

Scheduling

Author: Yasithzz | 2026

03 -- Scheduling

LifeMate Project -- English Documentation

1. What is the Scheduling Module?

The Scheduling module automatically generates a personalised weekly schedule (Monday to Sunday) for each user based on their ML-predicted stress level and their saved profile preferences. Rather than giving every user an identical timetable, LifeMate adapts work blocks, break lengths, exercise intensity, and meal labels to the individual -- a user with "Very High" stress gets shorter work sessions and gentle yoga, while a user with "Very Low" stress gets long focus blocks and high-intensity exercise.

2. WeeklyScheduleService.java -- Line-by-Line Explanation

2.1 -- Class Declaration and Dependencies

```
@Service
public class WeeklyScheduleService {
    private final WeeklyScheduleRepository repo;
    private final HolidayLeaveRepository hlRepo;
```

@Service marks this as a Spring-managed bean. Two repositories are injected:

- * repo -- reads/writes WeeklySchedule documents in MongoDB.
- * hlRepo -- reads the user's registered holidays and leave days.

2.2 -- generateForCurrentWeek()

```
public WeeklySchedule generateForCurrentWeek(String userEmail, int stressIndex,
                                             String stressLevel, User user) {
    LocalDate monday = mondayOfCurrentWeek();
    repo.deleteAllByUserEmailAndWeekStartDate(userEmail, monday.format(FMT));
    return buildAndSave(userEmail, stressIndex, stressLevel, user, monday);
}
```

Called immediately after a lifestyle submission. It:

- * Finds this week's Monday date.
- * Deletes any pre-existing schedule document for this user and week (prevents duplicates).

* Calls buildAndSave() to build a fresh schedule and persist it.

2.3 -- getLatestOrGenerate()

```
List<WeeklySchedule> existing =  
    repo.findByUserEmailAndWeekStartDate(userEmail, monday.format(FMT));  
if (!existing.isEmpty()) {  
    if (existing.size() > 1) existing.stream().skip(1).forEach(repo::delete);  
    return existing.get(0);  
}  
return buildAndSave(userEmail, stressIndex, stressLevel, user, monday);
```

Used when the Schedule page loads. Returns the existing schedule if one already exists for this week; otherwise generates a new one. Handles edge cases where duplicate documents crept in by keeping only the first and deleting the rest.

2.4 -- updateSlotStatus()

```
slots.forEach(s -> { if (s.getItemId().equals(itemId)) s.setStatus(status); });  
ws.setUserModified(true);  
return repo.save(ws);
```

Called when the user clicks the complete/pending toggle on a schedule slot in the UI. Finds the slot by its UUID itemId, updates its status, flags the document as user-modified, and saves.

2.5 -- addCustomSlot()

```
slot.setItemId(UUID.randomUUID().toString());  
ws.getDays().computeIfAbsent(day, k -> new ArrayList<>()).add(slot);  
sortDay(ws.getDays().get(day));  
return repo.save(ws);
```

Called when the user adds a custom activity via the "Add Activity" form. Generates a unique UUID for the new slot, appends it to the correct day's list, re-sorts by start time, and saves.

2.6 -- removeSlot()

```
ws.getDays().getOrDefault(day, List.of()).removeIf(s -> s.getItemId().equals(itemId));  
return repo.save(ws);
```

Removes a slot from the schedule by matching its UUID and saves the updated document.

2.7 -- buildAndSave() -- Core Schedule Builder

```
List<String> weekDates = new ArrayList<>();
for (int i = 0; i < 7; i++) weekDates.add(monday.plusDays(i).format(FMT));
Set<String> leaveDates = new HashSet<>{
    hlRepo.findByUserEmailOrderByDateAsc(userEmail).stream()
        .map(h -> h.getDate()).filter(weekDates::contains).toList()};
```

Collects the 7 dates of the current week and filters them against the user's registered holidays to build a Set of leave dates.

```
Set<String> workingDaySet = new HashSet<>{
    user.getWorkingDays() != null && !user.getWorkingDays().isEmpty()
        ? user.getWorkingDays()
        : List.of("MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY")};
```

Reads the user's custom working days from their profile. Falls back to Monday-Friday if none are set.

```
ws.getDays().put(DAYS[i], isLeave
    ? leaveSlots()
    : buildDay(stressIndex, !isWorkingDay, user));
```

For each of the 7 days: if it is a leave/holiday date, the day gets a single "Rest Day / Holiday" slot. Otherwise buildDay() constructs a full personalised day.

2.8 -- buildDay() -- Stress-Adaptive Day Construction

```
String wake = nvl(user.getWakeTime(), "07:00");
String bed = nvl(user.getSleepSchedule(), "22:30");
String[] wWork = parseWorkHours(nvl(user.getWorkHours(), "09:00-17:00"));
String wp = nvl(user.getWorkoutPreference(), "Moderate");
String ft = nvl(user.getFreeTimePreference(), "Reading");
String mp = nvl(user.getMealPreference(), "Balanced");
```

Reads six user profile fields (with sensible defaults via nvl()). These drive the personalisation of every slot.

Stress-driven work block sizing:

```
int blockLen = stress >= 4 ? 45 : stress == 3 ? 75 : 90;
int breakLen = stress >= 4 ? 20 : stress == 3 ? 15 : 10;
```

Stress Level	Work Block	Break
Very High (4)	45 min	20 min
High (3)	75 min	15 min
Normal/Low/Very Low (0-2)	90 min	10 min

High stress means shorter focus sessions with longer recovery breaks.

Exercise selection:

```
private String exerciseName(String pref, int stress) {
    if (stress >= 4) return "Gentle Yoga / Breathing Exercises";
    if (stress == 3) return "Light Walk or Stretching";
    return switch (pref.toLowerCase()) {
        case "intense" -> "High-Intensity Workout (HIIT / Running)";
        case "light"   -> "Light Walk or Yoga";
        default        -> "Moderate Exercise (Cycling / Gym)";
    };
}
```

Stress level always overrides the user's preferred exercise type when stress is high. At normal or low stress the user preference governs.

Meal personalisation:

```
private String mealName(String pref, String mealType) {
    String base = switch (pref.toLowerCase()) {
        case "vegetarian","vegan" -> "Plant-Based ";
        case "high-protein"       -> "High-Protein ";
        case "keto"               -> "Keto ";
        case "mediterranean"      -> "Mediterranean-Style ";
        default                   -> "";
    };
    return base + mealType;
}
```

Prepends the user's dietary preference to the meal slot title. "Keto Breakfast", "Plant-Based Lunch", etc.

2.9 -- Time Arithmetic Helpers

```
private String addMins(String time, int mins) {
    int[] t = parseTime(time);
    int total = t[0]*60 + t[1] + mins;
    return fmt(total/60, total%60);
}
```

Converts a "HH:mm" string to total minutes, adds/subtracts, and formats back. All slot start and end times are computed with these helpers -- no hardcoded clock times in the schedule (except anchor

points like dinner and wind-down).

2.10 -- mondayOfCurrentWeek()

```
private LocalDate mondayOfCurrentWeek() {  
    return LocalDate.now().with(DayOfWeek.MONDAY);  
}
```

Returns the Monday of the current ISO week. All schedule documents are keyed on this date so the app always knows which week a schedule belongs to.

3. ScheduleRepository.java -- Explanation

```
public interface ScheduleRepository extends MongoRepository<Schedule, String> {  
    Optional<Schedule> findTopByUserEmailOrderByCreatedAtDesc(String userEmail);  
    Optional<Schedule> findByUserEmailAndScheduleDate(String userEmail, String scheduleDate);  
    void deleteByUserEmail(String userEmail);  
}
```

Spring Data MongoDB generates the implementation automatically from method names.

Method	Purpose
`findTopByUserEmail...Desc`	Gets the user's most recent schedule document
`findByUserEmailAndScheduleDate`	Gets a schedule for a specific date
`deleteByUserEmail`	Removes all schedules when the user account is deleted

4. Complete Scheduling Flow

```
User submits lifestyle data (LifestylePage)
  v
Backend calls ML Service POST /predict
  v stress_level, stress_index returned
Backend calls WeeklyScheduleService.generateForCurrentWeek()
  v
  1. mondayOfCurrentWeek() -- anchor date
  2. HolidayLeaveRepository -- load user leave dates for this week
  3. User profile -- wake, sleep, work hours, workout/meal/free-time prefs
  4. Loop 7 days:
      isLeave? -> leaveSlots() ["Rest Day / Holiday"]
      isWeekend? -> buildDay(stress, weekend=true, user)
      isWorkday? -> buildDay(stress, weekend=false, user)
          * Sleep slot (00:00 -> wakeTime)
          * Morning Routine (stress-length)
          * Breakfast (personalised meal label)
          * Work blocks (stress-length) + Breaks + Lunch
          * Exercise (stress + preference driven)
          * Dinner (personalised label)
          * Evening Leisure (user free-time preference)
          * Wind Down (stress-length)
          * Sleep (bedTime -> 23:59)
  5. WeeklySchedule saved to MongoDB
  v
Frontend SchedulePage fetches GET /api/schedule/weekly
  v
User sees Mon-Sun personalised schedule with progress tracking
```

Documentation -- LifeMate Project, Yasithzz, 2026